

REPORT - TASK 3

- I have chosen task-3 as my additional task. The given dataset is about different health features of patients to predict whether they have hypertension or not.
- I analysed the dataset using a jupyter notebook. First, I performed Exploratory Data Analysis (EDA) which included basic statistics. I used various commands such as head, tail, shape, describe, nunique, etc.
- I have generated a heatmap, pairplots, scatter plot and a boxplot of my features which shows trends and relationships.

- The next step was preprocessing of the dataset to make it ready for training the model. Here i did following steps
 1. I was to check for any null values. There were a lot of null values in the 'Medication' column of the dataset but I chose to completely drop this column because it required discrete values and wasn't providing much to the prediction of our outcome.
 2. The next step was encoding categorical features which I did using 'get dummies' in 'pandas'. This converted all my features with string values into separate columns each with a discrete value which is marked as 'True' or 'False' depending on which category the reading falls into.
 3. The last step of preprocessing was to scale the numerical features. Instead of subtracting all the values with their mean values, I used the minimum values instead. This was because using mean values would give us some negative values after scaling.
 4. Another step in preprocessing was outlier detection and handling of outliers. In salt_intake there were a couple of outliers on the upper side which I removed to make the dataset cleaner.
 5. The last step of preprocessing was splitting the data into a test-train. The split division selected was 80-20 using sklearn.

- Then I started building my decision tree classifier.
 - Starting with unpruned tree classifier of this dataset gave me an accuracy score of 1.0 for train and 0.962217 for test. This is overfitting because the model tried to fit each and every sample of

the dataset. But this kind of model may not be good with unseen new input.

- To further improve the accuracy I pruned the data and made a new decision tree using entropy. This method gave me an accuracy score of 1.0 for train and 0.966973 for test. This is better accuracy than an unpruned decision tree for test data.
 - I then further tried to build more decision tree models with different max_depth values (namely 6, 8 and 9). Also I tried with more specifications such as min_samples_split, min_samples_leaf and max_features to try and improve the accuracy.
 - However, the addition of these features decreased the accuracy. So the final conclusion was that the decision tree created at the second stage with only criteria of entropy was the best after validation. Reducing max_depth is not helping to improve accuracy here. With that I concluded about the best model here.
-
- In the next step I built the confusion matrix for the best model. The confusion matrix clearly shows the model has quite good accuracy for test data with very less false predictions. Also generated classification report with precision, recall, F1-score to show the efficiency of the model.
 - The last step is to find the most influencing features for a patient's risk of hypertension. The model gave 'BP_History', 'Salt_Intake', 'Age', 'BMI', 'Sleeping_Duration' as top 5 features which influence the outcome most. The results tell us that a combination of medical history, dietary habits, demographic factors, body composition and lifestyle habits are responsible for determining health risks such as hypertension.
-
- The first challenge I faced in this task was setting up a jupyter notebook. I have worked a little bit with pycharm before, but the jupyter notebook was something new and I wasn't aware of how to set it up. It took me quite some time to set it up but once setup was done, I found it pretty easy to understand it and write the code syntaxes, since I have learnt and worked in python before.
 - Another problem I faced was working with jupyter's way of run command. Here, once a command is run it makes a permanent change in our dataset and if any mistake was found in the intermediate step then reverting it is

very difficult. Since I was using the same dataset after any preprocessing step (encoding, scaling, handling null etc) it is always difficult if I realized the mistake after some steps because there could be more operations performed on that dataset. I get rid of that by stopping the kernel and run all again but that may not be the best solution and I should find a better way of coding here. This is something I struggled with briefly.

- One way in which I made my models more accurate was to remove the “outliers”. Outliers are certain data points whose values are very far away from the usual trend. These cause unnecessary variance in the average values and confuse the model, which is why getting rid of them is seen as a better option.